

# Sisteme de operare

Laborator 5

Semestrul I 2025-2026

# Laborator 5

- controlul proceselor
- crearea proceselor
- executia programelor
- terminarea proceselor
- race conditions

# Executia programelor

- *fork* creeaza un proces nou, dar codul noului proces e o copie a codului parinte
- apelul sistem *exec* incarca de pe disc imaginea unui program nou care se va suprapune peste imaginea din memorie a procesului apelant
  - Obs: suprapunerea inseamna practic distrugerea procesului apelant  
ex: *\$ exec firefox*

- schema tipica de utilizare *exec*:

```

if(!fork()) {
    execl("/bin/ls", "ls", "-l", (char*)0);
    exit(1); // aici nu se ajunge niciodata, decat daca apelul exec a
            // esuat, motiv pt care apelam exit cu cod de eroare
}

// in mod uzual, aici procesul parinte continua singur, daca apelul exec din
// procesul copil a reusit

```

# Executia unui program

```
#include <unistd.h>

int execl(const char *pathname, const char *arg0, ... /* (char *) 0 */ );

int execv(const char *pathname, char *const argv[]);

int execlp(const char *pathname, const char *arg0, ...
           /* (char *) 0, char *const envp[] */ );

int execve(const char *pathname, char *const argv[], char *const envp[]);

int execlp(const char *filename, const char *arg0, ... /* (char *) 0 */ );

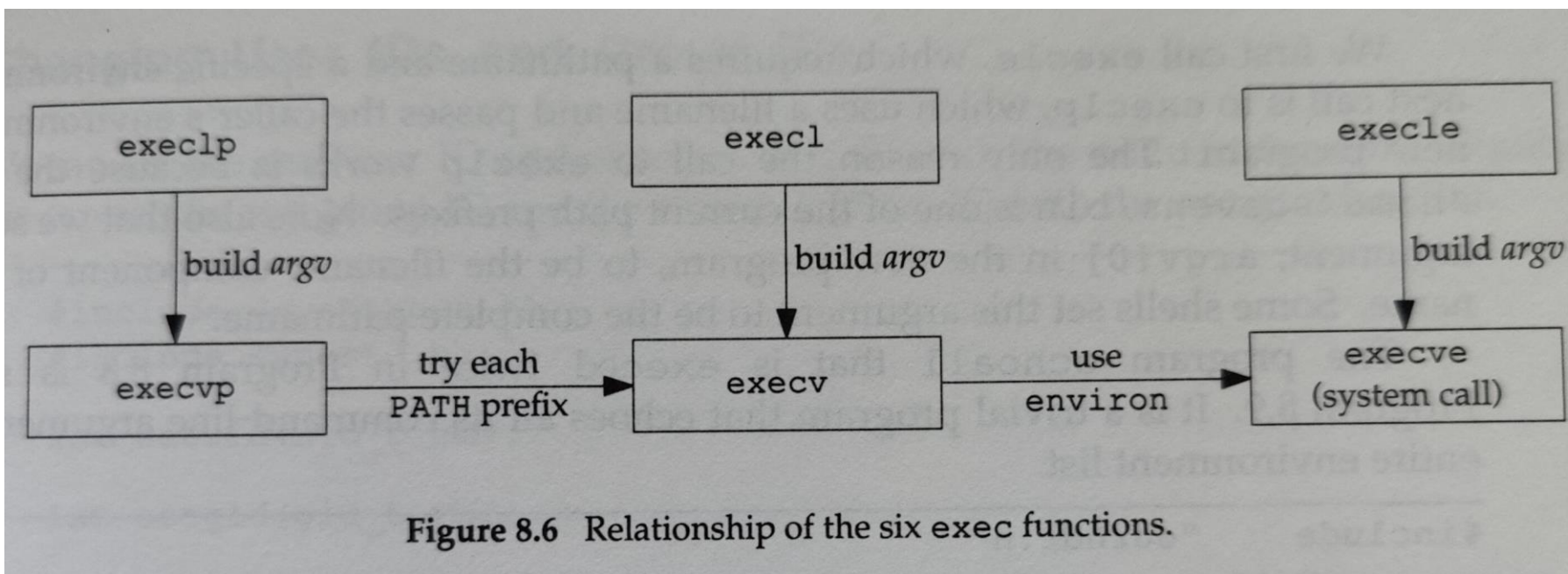
int execvp(const char *filename, char *const argv[]);
```

All six return: -1 on error, no return on success

# Diferente apeluri exec

- 6 rutine, doar una singura este apel sistem (*execve*)
- primele 4 folosesc calea absoluta a executabilului, ultimele doua recurg la variabila de mediu PATH pt a identifica executabilul
- exista doua feluri de a specifica parametrul noului program:
  - *lista de argumente*: *execl*, *execlp* si *execle* - lista parametrilor se incheie cu (char \*0) sau NULL pointer
  - *vector de argumente*: *execv*, *execvp*, *execve* necesita gruparea argumentelor intr-un vector
- mediul de executie poate fi furnizat:
  - *explicit*: *execve*, *execle* primesc ca parametru de apel un pointer catre lista de variabile de mediu
  - *implicit*: celelalte 4 apeluri al caror nume nu se termina cu "e" acceseaza variabilele de mediu prin intermediul variabilei globale *environ*  
*extern char\*\* environ;*

# Diferente apeluri exec (cont.)



# Proprietati mostenite de la procesul apelant

- dupa *exec*, se pastreaza urmatoarele informatii
  - PID si PPID
  - real UID/GID
  - GID-uri suplimentare
  - GID de proces
  - ID-ul de sesiune,
  - teminalul de control
  - CWD si root directory
  - timpul ramas din alarme
  - umask
  - masca de semnale + semnalele in asteptare
  - limitele referitoare la resurse (v. comanda shell *ulimit*)
  - diversi timpi de executie ai procesului
  - etc.

# Functia system

- uneori e util sa executam o comanda shell direct din program

ex: `system("ls -l > out");`

`int system(const char* cmd);`

- *system* este implementata folosind *fork+exec+waitpid*



# Script-uri shell

- fișiere text care conțin comenzi interpretate de un interpretor (eventual shell)
- uzual încep cu o linie de forma

`#! <cale interpretor comenzi> [arg optionale]`

ex: `#!/bin/bash`, `#!/usr/bin/python`

- kernelul apelează interpretorul de comenzi cu *fork-exec*
- interpretorul interpretează comenzile din fișierul text
- ex script *myscript.sh*

`#!/bin/bash`

`ls -l > out`

`cat out`

`exit 0`

# Script-uri shell (cont.)

- in functie de interpretor, script-urile pot fi programe in sine, cu instructiuni (comenzi de fapt) de tip if-then-else, for, switch, etc
- ex: programul Unix/Linux *awk* folosit de scriptul *myawk.sh* ca interpretor

```
#!/usr/bin/awk -f
```

```
BEGIN {
```

```
    for(i = 0; i < ARGV; i++)
```

```
        printf "ARGV[%d] = %s\n", i, ARGV[i]
```

```
    exit
```

```
}
```

# Executia script-urilor

- prin apelul direct al interpretorului potrivit

ex:        *\$ sh myscript.sh*

- prin setarea permisiunii de executie pt fisierul script

ex:        *\$ chmod u+x myscript.sh*

*\$ ./myscript.sh*

## Sisteme de operare

### Laborator 5

### Gestiunea/controlul proceselor (cont.)

1. Creati un nou subdirector *lab5/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab5/bin*.

2. Scrieti un program C **my-exec-engine.c** care primeste ca parametru in linie de comanda un program executabil si apeleaza *vfork* pentru a crea un nou proces. Procesul copil executa comanda primita ca argument folosind apelul sistem *execvp*. Parintele asteapta terminarea executiei copilului folosind apelul sistem *wait*. Ca exemplu, comanda trebuie sa functioneze in felul urmator:

```
$ my-exec-engine ls -l -a -R
```

3. Scrieti un program C **my-script-engine.c** care primeste ca parametru in linie de comanda un shell script. Programul parseaza prima linie a scriptului in cautarea interpretorului care trebuie lansat in executie. Odata identificat interpretorul, programul apeleaza *fork* pentru a crea un nou proces si apoi lanseaza in executie interpretorul (folosind *execv*) in contextul procesului copil pentru a executa scriptul. Parintele asteapta terminarea executiei copilului folosind apelul sistem *wait*.

Puteti folosi ca exemple scripturile de pe slide-urile de laborator, stocate in fisierele executabile *mymyscript*, respectiv *myawk* ca mai jos:

```
$ my-script-engine myscript
$ my-script-engine myawk
$ my-script-engine myawk 1 2 3 4 5
```

Obs: nu uitati ca interpretoarele de pe prima linie a unui shell script pot primi si parametri, asa cum este cazul scriptului *myawk* care apeleaza programul *awk* cu flagul *-f*.

4. Folositi experienta dobandita in exercitiile de mai sus pentru a scrie un program C **my-shell.c** care citeste in bucla comenzi de la tastatura si le executa. Mai exact, programul ruleaza intr-o bucla infinita in care afiseaza pe ecran promptul “my-shell> “, citeste datele introduse la tastatura si foloseste o functionalitate similara celei *my-exec-engine* pentru a executa comanda introdusa de utilizator. Programul se termina cand utilizatorul tasteaza Ctrl-d (adica EOF).

Pentru exemplificare, urmatoarele comenzi trebuie sa poate fi executate de **my-shell**:

```
$ my-shell
my-shell> ls -l
my-shell> my-exec-engine ls -l
my-shell> ./myawk 1 2 3 4 5
my-shell> my-script-engine myawk 1 2 3
my-shell> Ctrl-d
$
```

Extindeti apoi functionalitatea lui **my-shell** a.i. sa inteleaga redirectarea intrarii si iesirii standard, mai exact sa fie capabil sa execute comenzi ca mai jos:

```
$ my-shell
my-shell> ls -l > out
my-shell> my-exec-engine ls -l >> out
my-shell> cat < myawk
my-shell> my-exec-engine cat < myawk
my-shell> Ctrl-d
$
```

5. Scrieti un program C **system.c** care simuleaza comportamentul programului de la punctul 2 folosind functia de biblioteca *system*. Cum ati scrie varianta proprie a acestui program folosind *fork/exec*?

Indicatie: pentru a folosi functia *system* e nevoie sa concatenati string-urile furnizate ca parametri de apel ai programului (*argv[i]*). In acest sens, puteti folosi functia de biblioteca *strcat*.

6. Scrieti un program C **my-time.c** care simuleaza executia comenzii **time** si masoara timpii de executie ai unuia dintre programele de mai sus folosind apelul sistem *times*. Pentru referinta, executati comanda de mai jos:

```
$ time ls -l > /dev/null
```